

The U-Shaped Challenge of Language Grounding: How Instruction Granularity Induces VLA Shortcuts

Anonymous ACL submission

Abstract

Instruction granularity remains an uncontrolled variable in language-guided embodied AI, leaving its impact unknown. We propose an agent-centric formalization of granularity as rule width (w), a cross-task metric adapted from width-based planning. By measuring the peak concurrent state-feature dependencies an agent must resolve, we introduce w as a measure of instruction granularity and show that it correlates strongly with the decision-making burden neural agents face when grounding language. We also introduce MINI-BEHAVIOR-GRAN, a controlled benchmark systematically varying w across 20 tasks while holding perception and control constant. Operationalizing granularity as rule width (w) reveals a non-monotonic, U-shaped performance curve: while fine-grained instructions ($w \leq 1$) ease grounding, coarse ones ($w \geq 2$) induce “language-ignoring” shortcuts driven by high instruction predictability ($P(l|v)$). We further uncover a generalization asymmetry, where coarse-trained models transfer to fine instructions but not vice-versa, and show that mixed-granularity training is essential for robust language grounding.

1 Introduction

Despite the rapid progress in language-guided embodied AI, the community has yet to establish a consensus on a unified metric to quantify the granularity of instructions during language grounding. In this domain, language serves a specific functional role: it provides structural information to decompose complex tasks and reduce the search space of the underlying sequential decision-making process. Consequently, *instruction granularity* should not be conflated with superficial syntactic properties, such as sentence length or the number of entities mentioned; rather, it must rigorously reflect the *decision-making burden* that remains for the agent to resolve internally after grounding the language into the action space.

Existing research conceptualizes instruction granularity through two dominant lenses, both of which struggle to provide a consistent measure of the decision-making burden required for grounding. Studies in Natural Language Processing (NLP) typically operationalize granularity via surface-form properties, such as textual specificity or discourse structure (e.g., number of entities mentioned) (Dong and Lapata, 2018; Huang et al., 2025b). While these effectively measure syntactic complexity, they adopt a *text-centric* rather than *agent-centric* view. Consequently, they cannot quantify how much an instruction reduces an embodied agent’s *decision-making burden*.

From an *agent-centric* perspective, the essential function of instruction is to decompose complex tasks into tractable sub-problems, thereby reducing the planning burden (Drexler et al., 2022). Recent work in LLM-based agents thus treats granularity as *decomposition depth*—the extent to which an instruction enumerates subgoals versus stating only the final objective (Yang et al., 2024; Myers et al., 2024; Shi et al., 2025; Zhong et al., 2025). While this aligns with our view that *instruction granularity* modulates planning demand, decomposition depth remains an unstable, task-dependent proxy. It conflates instructional guidance with the task’s inherent structure: complex tasks with interleaved subgoals may resist breakdown (e.g., the Sussman Anomaly (Sussman, 1975)), while trivial ones can be artificially elongated.

To operationalize this, we draw inspiration from the concept of policy sketches (Drexler et al., 2024) and formalize instruction granularity as rule width (w) (Bonet et al., 2019). We hypothesize that w captures the decision-making burden that an instruction imposes, and we provide empirical evidence supporting this hypothesis. To this end, the contributions of our paper are:

(1) Width as a Formalism for Instruction Granularity: We formalize instruction granularity

as *rule width* (w), adapting a concept from width-based planning (Bonet et al., 2019; Drexler et al., 2024) that measures the peak number of concurrent state variables (features Φ) that must be tracked to fulfill the instructions. We argue w measures the *intrinsic decision complexity* of a task-instruction pair, applicable to both symbolic and neural agents (§ 3).

(2) A Controlled Benchmark: We present MINI-BEHAVIOR-GRAN, the first testbed to isolate instruction granularity from confounding factors like task horizon and perception. It enables the first study of how instruction granularity affects VLA performance (§ 4).

(3) The U-Shaped Trend and Width Cap: In this controlled benchmark, we identify a non-monotonic U-shaped relationship where performance peaks at the Fine ($w = 0$) and Coarse ($w \geq 2$) extremes. We also identify a performance collapse at high width ($w \geq 4$): agents fail to fulfill instructions requiring maintaining concurrent features ≥ 4 , revealing a sharp language grounding boundary in current VLA models (§ 5).

(4) Language-Ignoring Shortcuts: We diagnose the performance rebound at $w \geq 2$ as “Language Ignoring,” where agents bypass linguistic grounding to exploit visual shortcuts due to high instruction predictability ($P(l|v)$). We show that *Axial Training* (mixed-granularity) mitigates this failure mode. Additionally, we identify an *asymmetric generalization* pattern: agents trained on coarse instructions successfully transfer to fine-grained ones, but not vice versa.

2 Related Work

Instruction Abstraction and Granularity. In this work, *Instruction granularity* refers to the extent to which state and action constraints are explicitly specified versus left for the agent to infer when grounding language into actions. This is distinct from, yet often conflated with the concept of *linguistic abstraction*. The term *abstraction* is viewed as *representational compression*—using a compact description to stand for multiple instances or executions. This definition is common in prior work of computational thinking (Wing, 2010; Lachmy et al., 2022; Zheng et al., 2024). Importantly, granularity can vary independently of abstraction: “tidy the room” is coarse but non-abstract (it leaves the exact required actions underspecified, yet it lacks representational compression.), whereas “repeat wiping

until clean” is relatively specific yet abstract (it prescribes an action but uses a loop condition to compress multiple executions).

While not framed as the term “granularity”, we acknowledge that a rich body of work has focused on augmenting language specificity and its impact, for example through coarse-to-fine semantic parsing (Dong and Lapata, 2018; Li et al., 2020). Other work generates compositional instructions via constraint dropout and sub-task recombination (Huang et al., 2025b), or construct counterfactual instructions to augment data diversity (Glossop et al., 2025).

In embodied instruction following, varying instruction detail is often handled via a simple two-part decomposition: high-level plans are converted into sequences of low-level steps before being grounded into actions (Yang et al., 2024; Shi et al., 2025). Most recent VLA works, however, operate with static language instructions (Ma et al., 2025; Li et al., 2025). Some models like InstructVLA and ThinkAct explore having chain-of-thought reasoning steps before action decoding (Yang et al., 2025; Huang et al., 2025a). These hierarchical definitions remain task-specific and qualitative, lacking a unified, cross-task metric to establish shared granularity levels across tasks. Consequently, quantitatively controlled studies on the formal dynamics of instruction granularity remain scarce in the embodied AI domain.

Language Sensitivity vs. Ignoring in VLA. Recent VLA research highlights a dichotomy between fragile language sensitivity (Akula et al., 2022) and systematic language-ignoring (Xing et al., 2025). We reconcile these via rule width (w): low w (Fine) involves explicit feature constraints and low predictability ($P(l | v)$), which enforces the agent to rely on language cues. Conversely, $w \geq 2$ (Coarse) triggers ignoring via visual shortcuts as $P(l | v)$ increases. Crucially, we identify a hard capacity cap at $w \geq 4$, where agents fail to track concurrent state-features regardless of granularity, revealing a fundamental grounding boundary in current architectures.

3 Rule Width as Instruction Granularity

3.1 Running Example

We first illustrate *planning width* computation using a running example. Table 1 shows the desired state progressions of a task where two shoes are cleaned using a rag, and the rag is placed on the

State Progression	Concurrent Features Tracked (Sequence of Tuples)	Width
Initial State	$\{\langle \rangle\}$	0
Navigate to rag	$\{\langle p_r = X \rangle, \dots, \langle p_r = 0 \rangle\}$	1
Pick up rag	$\{\langle p_r = 0 \rangle, \langle H_r \rangle\}$	1
Navigate to sink	$\{\langle H_r, p_s = X \rangle, \dots, \langle H_r, p_s = 0 \rangle\}$	2
Turn on tap (sink)	$\{\langle H_r, p_s = 0 \rangle, \langle T_s \rangle\}$	2
Soak rag	$\{\langle H_r, T_s, p_s = 0 \rangle, \langle S_r \rangle\}$	3
Navigate to Shoe 1	$\{\langle H_r, S_r, p_{f1} \downarrow \rangle, \dots, \langle C_1 \rangle\}$	3
Clean Shoe 1	$\{\langle H_r, S_r, p_{f1} = 0 \rangle, \langle C_1 \rangle\}$	3
Navigate to Shoe 2	$\{\langle H_r, S_r, C_1, p_{f2} \downarrow \rangle, \dots, \langle C_2 \rangle\}$	4
Clean Shoe 2	$\{\langle H_r, S_r, C_1, p_{f2} = 0 \rangle, \langle C_1, C_2 \rangle\}$	4
Put rag on floor	$\{\langle C_1, C_2, H_r \rangle, \dots, \langle C_1, C_2, \text{onfloor}_r \rangle\}$	3

Table 1: State progression and width for the “Cleaning k Shoes” ($k = 2$). Width scales as $w = k + 2$.

floor afterwards. The key **environment dynamics** are: (1) the rag must be soaked before it can clean shoes, and (2) the tap (sink) must be turned on to soak the rag. We represent states using a set of task-relevant feature variables (Φ): (i) boolean features H_r (holding rag), T_s (tap (sink) is turned on), S_r (rag is soaked), C_i (shoe i is cleaned), onfloor_r (rag is on the floor); and (ii) spatial features p_r, p_s, p_f (distance to the rag, sink, and nearest uncleaned shoe, respectively).

First, approaching the rag is a navigation-only segment: the desired state progression is to decrease the distance feature p_r until $p_r = 0$. Thus, the width is 1 as only a single feature needs to be tracked. In this environment dynamics, $p_r = 0$ enables the state transition to H_r . Thus, the rag can be obtained at $p_r = 0$ and the width remains 1. Subsequently, H_r must be maintained while progressing to next states, resulting in a width of 2. The rag becoming soaked requires simultaneous maintenance of three features, i.e., at the sink ($p_s = 0$), holding the rag (H_r), and having the tap (sink) turned on (T_s), which yields a width of 3. After soaking, C_1 can be obtained by tracking H_r , S_r , and p_f , giving width 3. The subtlety arises when cleaning the second shoe: one has to track the same features as before, plus maintaining the cleanliness of the first shoe (C_1), which increases the width to 4.

Why no action model? Agents’ action sets constrain how state transitions are executed, but *width* focuses on state-level feature dependencies induced by the environment dynamics. Different agents (e.g., a rover vs. a drone) may have different action spaces to achieve the same feature change, so emphasizing state progressions promotes generalization of width-based concepts across different embodiments.

The impact of *instruction granularity* on VLA

behavior remains underexplored largely because the field lacks a **formal, quantifiable, cross-task** measure of granularity. Building on the limitations of heuristic metrics discussed in § 1, we formally equate *instruction granularity* with *latent planning demand*, i.e., the implicit decision-making burden required to ground text into executable actions. We operationalize this using *rule width* (w) by framing natural-language instructions as policy sketches (Drexler et al., 2024). Under this framework, fine-grained instructions decompose tasks into minimal-width sub-problems, whereas coarse instructions leave a large “grounding gap” that the agent must resolve internally.

3.2 Symbolic Task Representation

While VLAs operate on raw multimodal observations, we adopt a symbolic, relational state abstraction. Many embodied benchmarks provide such structured representations—object properties and relations—that can be expressed as *fluents* to form symbolic state or goal conditions (Shridhar et al., 2020; Li et al., 2024). Formally, we consider a classical planning problem $P = \langle \mathcal{F}, \mathcal{A}, I, G \rangle$, where $\mathcal{F}$ is a set of fluents, $\mathcal{A}$ a set of actions, I the initial state, and G the goal.

To bridge raw states and high-level instructions, we define a set of **features** $\Phi = \{\phi_1, \dots, \phi_n\}$, where ϕ_i is a Boolean or integer-valued function over state space S . These features serve as domain-level abstractions that capture task-relevant aspects of the environment. Thus, each s is associated with a **feature valuation vector** $\phi(s) = (\phi_1(s), \dots, \phi_n(s))$.

3.3 Instructions as Rule Sets $\mathcal{R}$

To study instruction granularity in a controlled manner, we formalize each instruction as a symbolic interpretation, represented as a set of “if-then” rules $\mathcal{R} = \{r_1, \dots, r_m\}$ defined over the feature space Φ . This rule-based representation does not aim to capture the full expressiveness of natural language; instead, it distills the constraint/goal structure that governs the computational cost incurred by a symbolic search algorithm (i.e., planning demand). Specifically, each r_i is of the form $C_{r_i} \mapsto E_{r_i}$:

- C_{r_i} (the *condition*) is a conjunction of feature constraints. For a Boolean feature p , a constraint is p or $\neg p$; for a numerical feature n , it is $n = 0$ or $n > 0$.

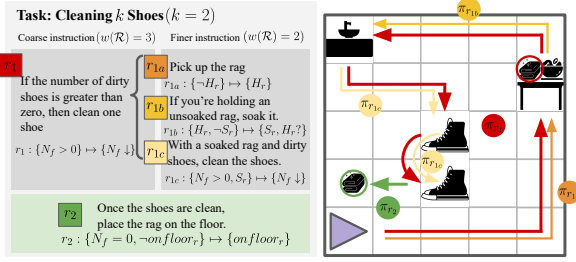


Figure 1: (Left) Factoring r_1 into r_{1a} - r_{1c} reduces latent planning complexity via explicit action guidance. (Right) Fine instructions segment the long-horizon coarse trajectory (π_{r_1} , red).

- E_{r_i} (the *effect*) is a conjunction of feature changes. For a Boolean feature p , an effect is p , $\neg p$, or $p?$ (allowing any change); for a numerical feature n , it is $n \downarrow$ (decrease), $n \uparrow$ (increase), or $n?$ (any change).

These forms capture the core semantics of the instructions we study. A rule r_i is **fulfilled** by a state sequence $(s, \dots, s')$ if $\phi(s) \models C_{r_i}$ and the pair $(\phi(s), \phi(s'))$ satisfies the changes in E_{r_i} (e.g., $\forall (n \downarrow) \in E_{r_i}, \phi_n(s') < \phi_n(s)$) while keeping unmentioned features constant (i.e., $\phi_j(s') = \phi_j(s)$ for $j \notin E_{r_i}$). To ensure that the agent’s performance strictly reflects its grounding ability rather than the unsolvability of the task, we guarantee that all instruction rule sets $\mathcal{R}$ are executable and dead-end-free. Formally, this is achieved by ensuring the rules are well-formed (i.e., they terminate at the goal) and satisfy the soundness and closure properties (Bonet and Geffner, 2021).

3.4 Rule Width as a Granularity Measure

The width $w(r_i)$ of a rule r_i represents the number of state variables (features) that must be tracked concurrently to fulfill the transition. Formally, for a rule to have width k , the agent must coordinate at most k features from Φ to reach the effect E_{r_i} from a state satisfying C_{r_i} (Lipovetzky and Geffner, 2012).

We define the granularity of an instruction $\mathcal{R}$ by its width-based planning bottleneck: the maximum width among its constituent rules: $w(\mathcal{R}) = \max_{r_i \in \mathcal{R}} w(r_i)$.

We continue the running example to demonstrate two instruction sets, each representing a different width level, with two variants visualized in Figure 1. We also append a **counting feature** in Φ to track the number of uncleaned shoes, denoted as N_f .

Coarse-Grained Instruction ($w(\mathcal{R}) = 3$) This instruction states only the high-level rules and their desired ordering, leaving the internal structure of each sub-task implicit:

- $r_1 : \{N_f > 0\} \mapsto \{N_f \downarrow\}$ (clean a shoe; $w = 3$)
- $r_2 : \{N_f = 0, \neg onfloor_r\} \mapsto \{onfloor_r\}$ (put the rag on the floor after cleaning shoes; $w = 1$)

Middle-Grained Instruction ($w(\mathcal{R}) = 2$) This variant decomposes r_1 to explicitly state the *soaking* process:

- $r_{1a} : \{\neg H_r\} \mapsto \{H_r\}$ (pick up the rag; $w = 1$)
- $r_{1b} : \{H_r, \neg S_r\} \mapsto \{S_r, H_r\}$ (soak the rag)
- $r_{1c} : \{N_f > 0, S_r\} \mapsto \{N_f \downarrow\}$ (clean; $w = 2$)
- $r_2 : \{N_f = 0, \neg onfloor_r\} \mapsto \{onfloor_r\}$ (put the rag on the floor; $w = 1$)

This rule set is well-formed: it is goal-terminating via r_2 , and the rules are sequentially reachable, meaning the successor state of any fulfilled rule (that is not the goal) also meets the condition of at least one rule in $\mathcal{R}$. The set is also robust to perturbations: if an agent drops the rag after soaking it ($\neg H_r$), the rule set remains applicable by reverting to r_{1a} to reacquire the rag before proceeding to the cleaning task (r_{1c}).

Width as a Planning Proxy. The fundamental purpose of decomposing a task into a rule set $\mathcal{R}$ is to provide structural scaffolding, guiding the agent through intermediate “stepping stone” states. By design, we assume the execution of r_i does not negate established fluents unless explicitly required by E_{r_i} . This is the essence of language-guided decomposition: the agent treats features in C_{r_i} as *maintained*, allowing working memory to focus on *incremental* features. Without this assumption, language guidance would offer no reduction in complexity; the agent would need to track every feature since the initial state.

This intuition is formally grounded in novelty-driven Breadth-First Search (Lipovetzky and Geffner, 2017). By pruning states that fail to achieve a new feature conjunction of size $\leq k$, search complexity is bounded by $O(|\Phi|^k)$. Thus, trajectories that unnecessarily negate an already-achieved precondition C_{r_i} (unless required by E_{r_i}) are longer than those preserving it, and are pruned from the optimal frontier. Consequently, features in C_{r_i} that are already true and not altered by E_{r_i} are

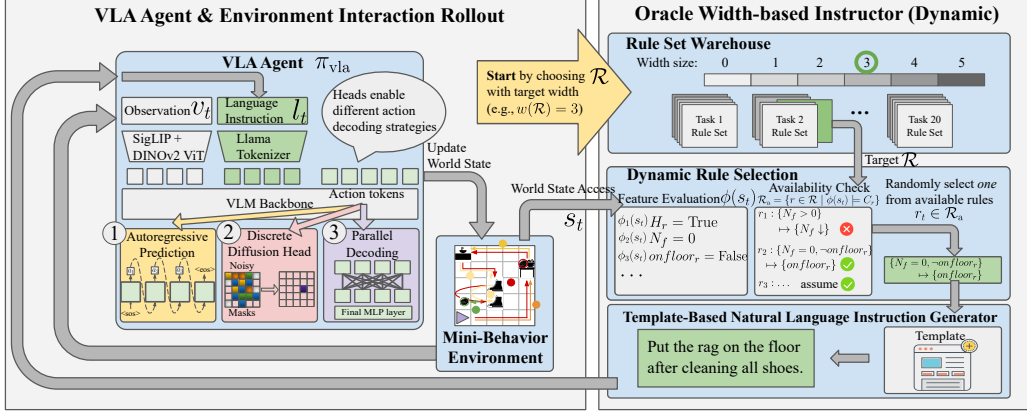


Figure 2: MINI-BEHAVIOR-GRAN Experimental Pipeline. We evaluate the impact of instruction granularity across three VLA action-decoding strategies. During inference time, an Oracle Instructor evaluates the current world state to decide available rule set $\mathcal{R}_a$ and active rule r_t . A rule-based natural language instruction l_t is generated accordingly. The process iterates until completion or max steps.

effectively maintained, and the active coordination w reduces to the novel features needed to reach the next subgoal. For instance, in r_{1b} (soaking), H_r is established by r_{1a} ; dropping it prematurely would merely lengthen the path to S_r , so active coordination is restricted to $\{p_s = 0, T_s\}$, yielding $w = 2$.

To communicate these rules to VLA agents, we translate each rule into a natural-language sentence via a template-based generator. These templates verbalize the features within each rule; e.g., $\{H_r, \neg S_r\} \mapsto \{S_r\}$ is realized as “If you are holding the rag but the rag is not yet soaked, soak the rag.” This construction preserves the granularity differences defined by the rule width. Additional language prompt examples are provided in § F.

4 Mini-BEHAVIOR-Gran Benchmark

To disentangle the influence of instruction granularity in VLA performance, we seek an environment that (1) retains long-horizon and heterogeneous activities so that instruction guidance meaningfully modulates the planning demand, while (2) simplifies perception and low-level control to avoid confounding factors. As such, we use MINI-BEHAVIOR (Jin et al., 2023) as our testbed. Detailed justification for our choice over other environments is provided in § B.

We introduce MINI-BEHAVIOR-GRAN, the first VLA benchmark specifically designed to enable a controlled study and formal quantification of instruction granularity via *rule width*. Specifically, we define a shared, domain-level feature set

Φ that captures task-relevant attributes across all 20 complex household tasks on top of the symbolic state abstraction from the original benchmark. For each task, we design rule sets with varying widths (up to six levels). We verify that all rule sets are well-formed and solvable. We utilize an Oracle Instructor to deliver instructions during inference. This ensures that any observed performance drops are strictly attributable to the model’s inability to ground specific granularity levels, rather than stochastic variations in instruction quality or applicability that would confound our cross-granularity analysis (see Figure 2 right panel). At each time t , it evaluates the current feature vector $\phi(s_t)$, identifies the set of available rules $\mathcal{R}_a = \{r \in \mathcal{R} \mid \phi(s_t) \models C_r\}$. We sample an active rule $r_t \in \mathcal{R}_a$ and it is then processed by a template-based translator to generate a natural language instruction l_t , to match the VLA model’s input format. l_t is paired with the current observation v_t and fed into the VLA model to predict action chunk $(a_t, \dots, a_{t+k}) \sim \pi_{\text{vla}}(\cdot \mid v_t, l_t)$.

5 Experiments

5.1 Experimental Setup

Task Suite & Training. We use MINI-BEHAVIOR-GRAN, which contains 20 complex tasks sharing the same feature set Φ . We further classify instruction granularity into three *relative* classes of *Fine* (F), *Medium* (M), and *Coarse* (C) via per-task quantile binning. In most cases, this mapping corresponds to $w = 0$ for Fine, $w = 1$ for Medium, and $w \geq 2$ for Coarse. We evaluate seven training settings: (1) **Pure F/M/C**

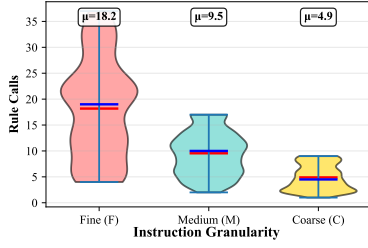


Figure 3: Rule call distribution across instruction granularities. # of rule calls decreases with coarser instructions. It decouples instruction length from width.

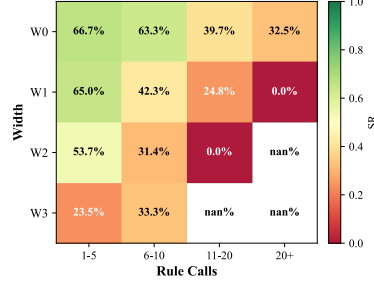


Figure 4: Impact of width and horizon on SR. SR decays with task horizon; larger instruction width imposes an additional burden even when horizon is fixed.

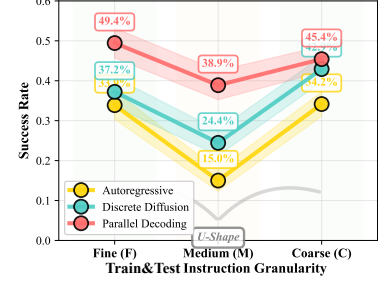


Figure 5: In-Distribution (matched train/test) SR across granularities (i.e., train Fine $\rightarrow$ test Fine, etc.). A non-monotonic U-shaped trend appears.

(1 : 0 : 0): trains exclusively on one granularity class; (2) **Centroid** (1 : 1 : 1): uniformly sample from all three classes; (3) **Axial F/M/C** (3 : 1 : 1): emphasizes one class with some exposure to the other classes. Our dataset provides 50 training and 10 evaluation instances per task, each paired with reference trajectories generated by BFWS width-based planner from Lipovetzky and Geffner (2017) (see § E for details).

Action-Decoding Archetypes. The primary architectural difference among latest VLA models lies in their action-decoding strategies (Zhong et al., 2025). Hence, we employ a unified VLM backbone but vary three representative action-decoding strategies (see Figure 2 left panel): (1) **Autoregressive** (AR), exemplified by RT-2 (Zitkovich et al., 2023), OpenVLA (Kim et al., 2024) and others (Reed et al., 2022; Huang et al., 2024; O’Neill et al., 2024) that predict actions sequentially via causal attention; (2) **(Discrete) Diffusion**, which uses diffusion process for action chunk prediction (Black et al., 2025; Bjorck et al., 2025; Shukor et al., 2025). We employ discrete diffusion (Liang et al., 2025) for compatibility with the discrete action space of our testbed; (3) **Parallel Decoding**, utilizing bidirectional attention for single-pass action prediction to improve throughput, as demonstrated by OpenVLA-OFT (Kim et al., 2025b), PD-VLA (Song et al., 2025) and other variants (Yin et al., 2025; Wang et al., 2025; Xiao et al., 2025). Details on model hyperparameters, hardware and training schedules are in § G.

5.2 Establishing the Yardstick: Decoupling Horizon from Width

A critical confounder in evaluating VLA planning complexity is the task sequence length. However,

in language-guided execution, instructions inherently decompose long-horizon tasks into segments. Consequently, the relevant measure of sequential burden shifts from the raw atomic horizon (total low-level action steps) to the instruction horizon, i.e., the number of rule calls issued by the instructor.

To strictly isolate rule width (w) from sequential burden, we train all agents on the same set of tasks. This ensures that the distribution of atomic horizons (total low-level steps) encountered during training is held constant across agents, leaving the number of rule calls and instruction granularity as variables: fine-grained instructions ($w = 0$) aggressively decompose the task into numerous low-width rule calls, resulting in a low average steps per rule call; coarse instructions ($w \geq 2$), by contrast, yield fewer rule calls with a higher average steps per call (Figure 3). To assess whether w serves as an effective indicator of latent planning demand for learning-based agents, we analyze Figure 4 using the *Centroid* configuration, which ensures balanced exposure across all granularity levels. The heatmap reveals a strict monotonic decay in Success Rate (SR) as rule width increases. Also, for a fixed w , SR decays as rule calls increase, mirroring greater sequential horizon complexity. This confirms w as a horizon-independent measure of the intrinsic planning demand imposed by instruction granularity.

5.3 The U-Shaped Paradox and Capacity Cap

To establish a baseline, we train models using the *Pure F/M/C* setting. the models are trained and evaluated on the same granularity class. As shown in Figure 5, all VLA archetypes exhibit a non-monotonic *U-shaped performance curve*.

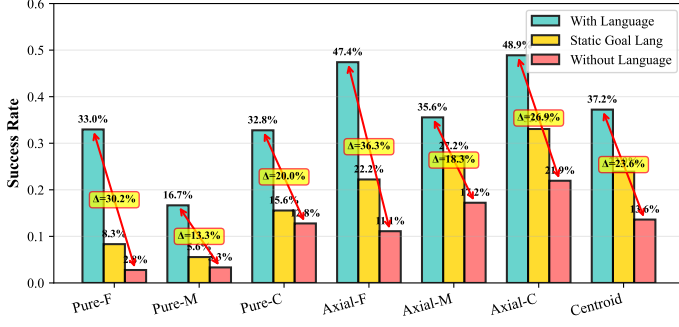


Figure 6: Smaller SR gaps indicate a reliance on vision-dominant, language-ignoring policies. The Pure-C setting exhibits negligible performance variance between (w/ goal lang) and (w/o lang).

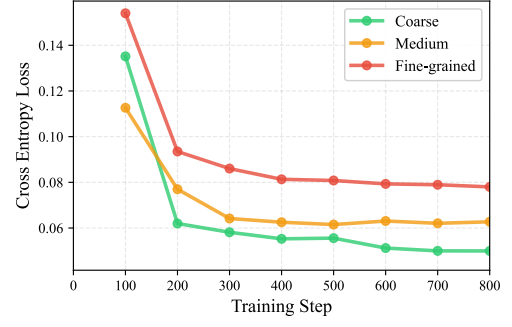


Figure 7: VLM instruction-prediction Cross Entropy (CE) loss across granularities. Larger CE loss indicates lower $P(l | v)$.

At the fine extreme ($w = 0$), where instructions require minimal latent reasoning to be grounded into actions, models achieve high performance. Performance then decreases at medium granularity ($w = 1$), before unexpectedly rebounding at the coarse regime ($w \geq 2$). This contradicts the assumption that higher planning width should monotonically degrade learning.

5.4 Mechanistic Probe: The Grounding Valley

Why does performance rebound at $w \geq 2$ while? We show this arises from increasing *instruction predictability* $P(l|v)$, which enables agents to bypass language and exploit visual-dominant policy.

Behavioral Evidence of Modality Collapse.

Following prior work that uses language ablation to diagnose modality reliance (Fei et al., 2025), we measure $\Delta\text{SR}_{\text{full} - \text{w/o}}$ in Figure 6. Fine-trained policies collapse without language (36.3% drop), while coarse-trained ones show a smaller 20.0% gap. Pure-C exhibits a negligible 2.8% difference between static goal descriptions (15.6%) and no language at all (12.8%). This confirms that VLA tends to learn a purely vision-dominant, reactive policies.

Why Language Becomes Ignorable. This behavioral shift suggests coarse instructions are simply *easier to predict from vision*. Formally, consider a surjective mapping f from fine-grained labels L_{fine} to coarser ones L_{coarse} . The conditional probability of a coarse label aggregates its fine-grained constituents: $P(c_j|v) = \sum_{l_i \in f^{-1}(c_j)} P(l_i|v)$. Thus coarse instructions are statistically more predictable from visual context. We confirm this predictability empirically by training a VLM (Qwen3-VL-4B via LoRA) to act as an trained instructor, predicting instructions from

visual observations. As Figure 7 shows, Cross-Entropy loss decreases monotonically with coarseness (Fine: 0.078, Medium: 0.063, Coarse: 0.047), a relative drop of 40% from Fine to Coarse. Together, these results form a complementary chain: probability aggregation explains *why* $P(l|v)$ increases, CE loss empirically confirm the increase, and ΔSR quantifies the language ignoring behavior.

Thus, we attribute this U-shaped behavior to how the agent dynamically shifts its reliance on text versus vision across different rule widths:

- **Fine** ($w = 0$): *Successful Grounding*. Low $P(l|v)$ (high CE loss, Figure 7) means language provides unique information. Combined with low-width grounding burden (§ 5.2), the model reliably maps instructions to actions, yielding high SR (Figure 5).
- **Medium** ($w = 1$): *The Alignment Trap*. As $P(l|v)$ rises, language becomes partially redundant, and the model reduces its reliance on text (ΔSR drops). However, unlike the coarse setting where language can be safely ignored, here language remains just useful enough to prevent the model from fully committing to a vision-only policy. The agent falls into a representational gap: language is hard to ground, yet still too present to abandon.
- **Coarse** ($2 \leq w \leq 3$): *Modality Collapse*. High $P(l|v)$ (Figure 7) causes the model to exploit visual shortcuts: ΔSR approaches 20% (Figure 6), which is a signature of modality collapse.
- **Capacity Cap** ($w \geq 4$): *Cognitive Overload*. Tasks requiring ≥ 4 concurrent features are provably hard for width-based planners (complexity $O(|\Phi|^4)$ (Lipovetzky and Geffner, 2017)). We

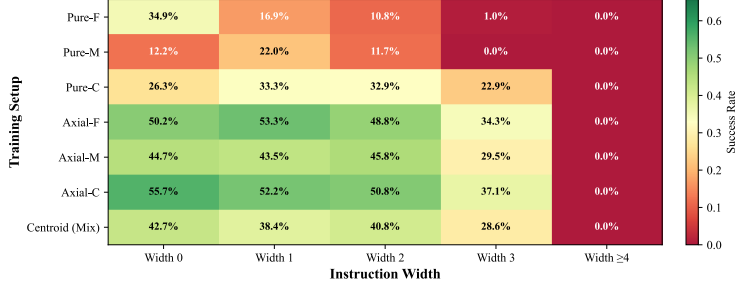


Figure 8: The x-axis are test instruction width, y-axis are training setup. Transfer is asymmetric: train coarse $\rightarrow$ test fine works, but train fine $\rightarrow$ test coarse does not. Mixed-granularity improves overall.

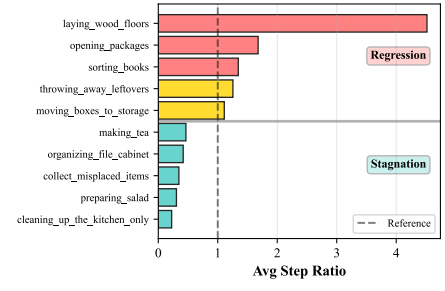


Figure 9: Low-width but long task horizon (e.g., laying woods) tends to cause regression, while high-width stagnates.

find VLA models hit the same ceiling: performance uniformly collapses at $w \geq 4$ (Figure 8). This alignment suggests the intrinsic difficulty captured by w translates directly to neural agents, mirroring the theoretical hardness of width-based planning.

Mitigating Modality Collapse via Axial Training.

Having diagnosed language-ignoring as a byproduct of high predictability, we hypothesize that increasing the diversity of the training language space L should artificially lower the average $P(l | v)$, thereby forcing the agent to re-engage with the text. Our mixed-granularity training results confirm this mechanism. All mixed-granularity models exhibit substantially larger ΔSR gaps compared to their *Pure-X* counterparts, indicating a successful reduction in language ignoring. Among these, the fine-weighted mixture *Axial-F* emerges as the optimal structural compromise: it achieves an overall success rate comparable to the coarse-heavy *Axial-C*, while maintaining a significantly larger ΔSR .

5.5 Other Results

Asymmetric Generalization. We observe a strict directional asymmetry (Figure 8): coarse-trained models successfully zero-shot transfer to fine texts, but fine-trained models fail on coarse texts. This corroborates our earlier ΔSR analysis: coarse training induces a vision-biased policy that generalizes robustly across instruction granularities. Conversely, fine training induces an over-reliance on dense instruction guidance, making the policy brittle when such guidance is withdrawn.

Failure Modes. Agent failure distributions bifurcate strictly with width (Figure 9). Low-width, long-horizon tasks (e.g., laying wood floors) predominantly suffer from *regression*: the agent violates previously satisfied fluents and cycles back to

prior states, reflecting subgoal serialization failures. High-width tasks ($w \geq 3$), by contrast, exhibit *stagnation*: the agent freezes, unable to resolve concurrent feature dependencies ($O(|\Phi|^w)$) required to initiate a valid state transition. This dichotomy mirrors the width-based complexity landscape (see § H for details).

6 Conclusion

This work introduces rule width (w) as a cross-task formalism for instruction granularity and demonstrates that it quantifies the state-tracking burden and correlates with language grounding difficulty in neural agents. We observed a non-monotonic relationship between granularity and performance, driven by a trade-off between instruction predictability ($P(l|v)$) and latent planning complexity. We demonstrate that mixed-granularity training mitigates modality collapse and preserves language grounding without sacrificing the robust visual reasoning skills learned from coarse trajectories.

7 Limitations

While this work systematically isolates the impact of language granularity on VLA planning, several boundaries constrain its current scope:

Discrete Action Spaces. We evaluate our framework in environments with discrete action spaces, a common abstraction in language-conditioned embodied AI (e.g., ALFWorld-based agent (Xiong et al., 2025; Kim et al., 2025a)) and LLM-based agent systems (Qin et al., 2024; Patil et al., 2024), where high-level reasoning is decoupled from low-level execution via skill primitives or API calls. This choice is intentional: it isolates symbolic planning from continuous control noise, allowing us to cleanly attribute observed effects to instruction granularity. However, extending this granularity-

aware framework to hybrid discrete-continuous or continuous action spaces remains essential for real-world robotics applications, where low-level motor commands interact with high-level linguistic abstractions.

Predefined State Features (Φ). Our formulation of rule width relies on a predefined set of general environmental predicates. While this set captures commonsense knowledge for household activities, it assumes task-relevant state abstractions are given a priori. Future work should explore developing learnable, hierarchical feature modules that dynamically construct task-relevant state abstractions would allow the model to adjust its own “internal granularity”.

Static Visual Granularity. As a controlled study of instruction granularity, we hold visual granularity constant across conditions. Yet visual perception itself inherently operates at multiple levels of abstraction from raw pixels to object segmentation images to scene graphs. Investigating the cross-modal interplay between visual representation granularity and instruction width is a critical next step.

References

Arjun R. Akula, Spandana Gella, Aishwarya Padmakumar, Mahdi Namazifar, Mohit Bansal, Jesse Thomason, and Dilek Hakkani-Tur. 2022. ALFRED-L: investigating the role of language for action learning in interactive visual environments. In *EMNLP*, pages 9369–9378. Association for Computational Linguistics.

Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, and 1 others. 2025. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*.

Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, brian ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, and 16 others. 2025. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 17–40. PMLR.

Blai Bonet, Guillem Francès, and Hector Geffner. 2019. Learning features and abstract actions for computing

generalized plans. In *AAAI*, pages 2703–2710. AAAI Press.

Blai Bonet and Hector Geffner. 2021. General policies, representations, and planning width. In *AAAI*, pages 11764–11773. AAAI Press.

Li Dong and Mirella Lapata. 2018. [Coarse-to-fine decoding for neural semantic parsing](#). In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 731–742, Melbourne, Australia. Association for Computational Linguistics.

Dominik Drexler, Jendrik Seipp, and Hector Geffner. 2022. Learning sketches for decomposing planning problems into subproblems of bounded width. In *ICAPS*, pages 62–70. AAAI Press.

Dominik Drexler, Jendrik Seipp, and Hector Geffner. 2024. Expressing and exploiting subgoal structure in classical planning using sketches. *J. Artif. Intell. Res.*, 80.

Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. 2025. [Libero-plus: In-depth robustness analysis of vision-language-action models](#). *Preprint*, arXiv:2510.13626.

Catherine Glossop, William Chen, Arjun Bhorkar, Dhruv Shah, and Sergey Levine. 2025. [Cast: Counterfactual labels improve instruction following in vision-language-action models](#). *Preprint*, arXiv:2508.13446.

Danijar Hafner. 2021. Benchmarking the spectrum of agent capabilities. *arXiv preprint arXiv:2109.06780*.

Chi-Pin Huang, Yueh-Hua Wu, Min-Hung Chen, Yu-Chiang Frank Wang, and Fu-En Yang. 2025a. [Thinkact: Vision-language-action reasoning via reinforced visual latent planning](#). *Preprint*, arXiv:2507.16815.

Hui Huang, Jiaheng Liu, Yancheng He, Shilong Li, Bing Xu, Conghui Zhu, Muyun Yang, and Tiejun Zhao. 2025b. Musc: Improving complex instruction following with multi-granularity self-contrastive training. In *ACL (1)*, pages 10667–10686. Association for Computational Linguistics.

Jiangyong Huang, Silong Yong, Xiaojian Ma, Xiongkun Linghu, Puhao Li, Yan Wang, Qing Li, Song-Chun Zhu, Baoxiong Jia, and Siyuan Huang. 2024. An embodied generalist agent in 3d world. In *ICML*.

Emily Jin, Jiaheng Hu, Zhuoyi Huang, Ruohan Zhang, Jiajun Wu, Li Fei-Fei, and Roberto Martín-Martín. 2023. [Mini-BEHAVIOR: A procedurally generated benchmark for long-horizon decision-making in embodied AI](#). In *NeurIPS 2023 Workshop on Generalization in Planning*.

743	Siddharth Karamcheti, Suraj Nair, Ashwin Balakrishna,	Xuanlin Li, Kyle Hsu, Jiayuan Gu, Oier Mees, Karl	800
744	Percy Liang, Thomas Kollar, and Dorsa Sadigh. 2024.	Pertsch, Homer Rich Walke, Chuyuan Fu, Ishikaa	801
745	Prismatic vlms: Investigating the design space of	Lunawat, Isabel Sieh, Sean Kirmani, Sergey Levine,	802
746	visually-conditioned language models . In <i>Forty-</i>	Jiajun Wu, Chelsea Finn, Hao Su, Quan Vuong, and	803
747	<i>first International Conference on Machine Learning,</i>	Ted Xiao. 2024. Evaluating real-world robot manipu-	804
748	<i>ICML 2024, Vienna, Austria, July 21-27, 2024</i> .	lation policies in simulation. In <i>CoRL</i> , volume 270	805
		of <i>Proceedings of Machine Learning Research</i> , pages	806
749	Jeonghye Kim, Sojeong Rhee, Minbeom Kim, Do-	3705–3728. PMLR.	807
750	hyung Kim, Sangmook Lee, Youngchul Sung, and		
751	Kyomin Jung. 2025a. ReflAct: World-grounded deci-	Zechang Li, Yuxuan Lai, Yansong Feng, and Dongyan	808
752	sion making in LLM agents via goal-state reflection .	Zhao. 2020. Domain adaptation for semantic pars-	809
753	In <i>Proceedings of the 2025 Conference on Empiri-</i>	ing . In <i>Proceedings of the Twenty-Ninth Interna-</i>	810
754	<i>cal Methods in Natural Language Processing</i> , pages	<i>tional Joint Conference on Artificial Intelligence, IJ-</i>	811
755	33433–33465, Suzhou, China. Association for Com-	<i>CAI 2020</i> , pages 3723–3729. ijcai.org .	812
756	putational Linguistics.		
757	Moo Jin Kim, Chelsea Finn, and Percy Liang.	Zhixuan Liang, Yizhuo Li, Tianshuo Yang, Chengyue	813
758	2025b. Fine-tuning vision-language-action mod-	Wu, Sitong Mao, Liuaio Pei, Xiaokang Yang, Jiang-	814
759	els: Optimizing speed and success. <i>arXiv preprint</i>	miao Pang, Yao Mu, and Ping Luo. 2025. Discrete	815
760	<i>arXiv:2502.19645</i> .	diffusion vla: Bringing discrete diffusion to action	816
		decoding in vision-language-action policies. <i>arXiv</i>	817
		<i>preprint arXiv:2508.20072</i> .	818
761	Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti,	Nir Lipovetzky and Hector Geffner. 2012. Width and	819
762	Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael	serialization of classical planning problems. In <i>ECAI</i> ,	820
763	Rafailov, Ethan Paul Foster, Pannag R. Sanketi, Quan	volume 242 of <i>Frontiers in Artificial Intelligence and</i>	821
764	Vuong, Thomas Kollar, Benjamin Burchfiel, Russ	<i>Applications</i> , pages 540–545. IOS Press.	822
765	Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang,		
766	and Chelsea Finn. 2024. Openvla: An open-source	Nir Lipovetzky and Hector Geffner. 2017. Best-first	823
767	vision-language-action model. In <i>CoRL</i> , volume 270	width search: Exploration and exploitation in clas-	824
768	of <i>Proceedings of Machine Learning Research</i> , pages	sical planning. In <i>AAAI</i> , pages 3590–3596. AAAI	825
769	2679–2713. PMLR.	Press.	826
770	Royi Lachmy, Valentina Pyatkin, Avshalom Manevich,	Yueen Ma, Zixing Song, Yuzheng Zhuang, Jianye	827
771	and Reut Tsarfaty. 2022. Draw me a flower: Process-	Hao, and Irwin King. 2025. A survey on vision-	828
772	ing and grounding abstraction in natural language.	language-action models for embodied ai . <i>Preprint</i> ,	829
773	<i>Trans. Assoc. Comput. Linguistics</i> , 10:1341–1356.	arXiv:2405.14093 .	830
774	Chengshu Li, Fei Xia, Roberto Martín-Martín, Michael	Vivek Myers, Chunyuan Zheng, Oier Mees, Kuan Fang,	831
775	Lingelbach, Sanjana Srivastava, Bokui Shen, Kent El-	and Sergey Levine. 2024. Policy adaptation via lan-	832
776	liott Vainio, Cem Gokmen, Gokul Dharan, Tanish	guage optimization: Decomposing tasks for few-shot	833
777	Jain, Andrey Kurenkov, Karen Liu, Hyowon Gweon,	imitation. In <i>CoRL</i> , volume 270 of <i>Proceedings</i>	834
778	Jiajun Wu, Li Fei-Fei, and Silvio Savarese. 2022a.	of <i>Machine Learning Research</i> , pages 1402–1426.	835
779	igibson 2.0: Object-centric simulation for robot learn-	PMLR.	836
780	ing of everyday household tasks . In <i>Proceedings of</i>		
781	<i>the 5th Conference on Robot Learning</i> , volume 164	Abby O’Neill, Abdul Rehman, Abhiram Maddukuri,	837
782	of <i>Proceedings of Machine Learning Research</i> , pages	Abhishek Gupta, Abhishek Padalkar, Abraham	838
783	455–465. PMLR.	Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar,	839
784	Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gok-	Ajinkya Jain, Albert Tung, Alex Bewley, Alexander	840
785	men, Sanjana Srivastava, Roberto Martín-Martín,	Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai,	841
786	Chen Wang, Gabrael Levine, Michael Lingelbach,	Anchit Gupta, Andrew E. Wang, Anikait Singh, and	842
787	Jiankai Sun, Mona Anvari, Minjune Hwang, Man-	260 others. 2024. Open x-embodiment: Robotic	843
788	asi Sharma, Arman Aydin, Dhruva Bansal, Samuel	learning datasets and RT-X models : Open x-	844
789	Hunter, Kyu-Young Kim, Alan Lou, Caleb R.	embodiment collaboration. In <i>ICRA</i> , pages 6892–	845
790	Matthews, and 9 others. 2022b. BEHAVIOR-1K:	6903. IEEE.	846
791	A benchmark for embodied AI with 1, 000 everyday	Maxime Oquab, Timothée Darcet, Théo Moutakanni,	847
792	activities and realistic simulation. In <i>CoRL</i> , volume	Huy V. Vo, Marc Szafraniec, Vasil Khalidov,	848
793	205 of <i>Proceedings of Machine Learning Research</i> ,	Pierre Fernandez, Daniel Haziza, Francisco Massa,	849
794	pages 80–93. PMLR.	Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas,	850
795	Wei Li, Renshan Zhang, Rui Shao, Jie He, and Liqiang	Wojciech Galuba, Russell Howes, Po-Yao Huang,	851
796	Nie. 2025. Cogvla: Cognition-aligned vision-	Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu	852
797	language-action model via instruction-driven routing	Sharma, and 7 others. 2024. Dinov2: Learning robust	853
798	& sparsification. <i>Advances in neural information</i>	visual features without supervision. <i>Trans. Mach.</i>	854
799	<i>processing systems</i> .	<i>Learn. Res.</i> , 2024.	855

Justification for using MINI-BEHAVIOR over alternatives (e.g., iGibson, CALVIN) based on controllability, symbolic abstraction, and isolation of language-grounding challenges.

2. Symbolic Representation of the Embodied Task (§ C) Formal definition of states, actions, and planning problems using classical STRIPS/PDDL semantics.

3. Details of the MINI-BEHAVIOR Environment (§ D) Overview of scene layout, object types, action space, and perception model.

4. Details of the MINI-BEHAVIOR-GRAN Extension (§ E) Procedure for generating multi-granularity instructions, details on the feature set, oracle instructor, and dataset construction.

5. Rule Set and Natural Language Prompt Examples (§ F) Details of the rule set bank of 20 tasks and sample generated instructions at different granularities.

6. Implementation Details (§ G) Model architecture (Prismatic-7B backbone, SigLIP+DINOv2 visual encoders), training hyperparameters, optimizer settings. VLM Predictor architecture and training details.

7. Additional Experimental Results (§ H) Extended analysis and ablations.

B Anticipatory defense on the choice of test environments and metrics

In this work, we selected Mini-BEHAVIOR as our testbed. This choice is motivated by several key considerations aligned with our research objectives. Our research objective is not to solve the full stack of robotic embodiment (which includes perception, continuous control, and physics), but specifically to isolate and analyze the impact of instruction granularity on language grounding and long-horizon planning.

Therefore, high-fidelity simulators (e.g., iGibson (Li et al., 2022a), BEHAVIOR-1K (Li et al., 2022b)) introduce significant noise through complex texture rendering and continuous motor control. While realistic, these factors act as confounders. When an agent fails in such environments, it is often difficult to attribute the failure to a lack of linguistic understanding (granularity issues) or a failure in low-level execution (e.g., grasping physics or occlusion). Mini-BEHAVIOR

retains the high-level decision complexity of household tasks—long horizons, multi-object interactions, and state dependencies—while abstracting away low-level sensorimotor noise.

In the perspective of fast prototyping and iterative experimentation, we have to also consider the rendering and physics simulation costs. Mini-BEHAVIOR’s grid-based discrete control and simplified visual representation allow for rapid data generation and model training.

Another consideration is the diversity of the task structures so that we can systematically vary instruction granularity across a wide range of scenarios. As such, Crafter (Hafner, 2021), which focuses on survival in a single domain, lacks the diverse task structures needed to obtain robust insights about instruction granularity.

Table 2 provides a detailed comparison of Mini-BEHAVIOR against other prominent benchmarks, highlighting why they were less suitable for this specific investigation.

C Symbolic Representation of the Embodied Task

We formalize the Mini-BEHAVIOR environment as a deterministic grid-based symbolic world. This allows us to represent the challenge as a classical planning problem $\mathcal{P} = \langle \mathcal{F}, \mathcal{A}, s_0, G \rangle$, where:

- $\mathcal{F}$ is a finite set of propositional fluents representing the state variables of the environment (e.g., object locations, states like *cooked* or *open*).
- $\mathcal{A}$ is a finite set of actions available to the agent.
- $s_0 \subseteq \mathcal{F}$ is the initial state configuration.
- $G \subseteq \mathcal{F}$ is the goal condition that must be satisfied.

A state $s \in \mathcal{S}$ is defined as a truth valuation over $\mathcal{F}$, with the state space $\mathcal{S} \subseteq 2^{\mathcal{F}}$ encompassing all valid combinations of fluent truth values.

D Details of MINI-BEHAVIOR Environment

To operationalize our formalism, we developed a **Universal PDDL Domain Model** (available at [minibehavior_gran_domain_model.pddl](#) in the codebase) capable of supporting all 20 household tasks without modification. This unified model comprises over 1,000 lines of PDDL 2.1 definitions and is fully compatible with the LAMA planner.

Table 2: Comparison of Embodied AI Environments regarding their suitability for studying Instruction Granularity.

Environment	Control	Horizon	Visuals	Symbolic Abstraction	Limitation
Mini-BEHAVIOR(*)	Discrete	Long	Grid	Support	Ideal balance: Supports symbolic state abstraction; diverse task suite; removes control noise while retaining logic complexity.
Crafter	Discrete	Long	Cartoon	Partial	Single task suite (survival); partially observable (memory heavy); few object types.
Meta-World	Continuous	Short	3D	No	Continuous control; no symbolic abstraction; focus on multi-task RL/manipulation, not long-horizon planning.
LIBERO	Continuous	Short	3D	No	Robotic arm focus; continuous control; no symbolic abstraction; VLA success rate already saturated (tasks relatively easy).
RoboTwin 2.0	Continuous	Short	3D	No	Continuous control; no symbolic abstraction; dual-arm manipulation focus; hard to segment expert trajectories and construct granular instructions.
CALVIN	Continuous	Medium	3D	No	Hard to design instruction granularity due to continuous action space (though provides diverse language); confounds planning with control difficulties.
ALFRED	Discrete	Long	3D Photo	Support	3D partially observable; poor infrastructure support for VLA training; high rendering cost.
iGibson / BEHAVIOR-1K	Continuous	Long	3D Photo	Partial	Good for long-horizon tasks, but 3D realistic vision context becomes a confounder.

We highly recommend checking the codebase for the complete domain model.

Table 3: Planning complexity in typical tasks.

Metric	Typical Range
Objects	15–40
Grid Locations	20–60
Ground Actions	10^3 – 10^5
Plan Length	50–200 primitive actions
Planning Time (LAMA)	0.5–300+ seconds

Search Space Characteristics

E Details of MINI-BEHAVIOR-GRAN Extension

MINI-BEHAVIOR provides the foundational task skeletons, which we extend in MINI-BEHAVIOR-GRAN to support multi-granularity instruction generation. A core component of this extension is the definition of a symbolic feature set used to generate rule-based policy sketches.

Table 4: Boolean State Features

Feature	Applicable Objects	Semantics
Holding	All Items	Is the agent holding the specified object?
isOpened	Cabinets, Drawers, Doors	Is the container/door open?
isToggled	Sinks, Stoves, Switches	Is the appliance turned on?
isSoaked	Rags, Sponges	Is the cleaning tool wet?
isCleaned	Shoes, Dishes, Floor	Is the object free of stains?
isDustFree	Furniture, Surfaces	Is the object free of dust?

E.1 Feature Categories

The environment defines two primary categories of features: Boolean State Features and Quantitative Features.

E.1.1 Boolean State Features

These features evaluate whether a specific object instance satisfies a predicate. They require grounding (binding to specific object instances, e.g., rag-01).

E.1.2 Quantitative and Spatial Features

These features capture distances, counts, and spatial relationships. Crucially, they support temporal comparison allowing conditions such as “distance decreased” or “count dropped to zero.”

- **DistanceToNearest(type, condition):** Measures the Manhattan distance from the agent

Table 5: Available Count Functions for Rule Definitions

Category	Function Description
State-Based	count_not_cleaned: Objects that are currently stained. count_not_soaked: Cleaning tools that are dry. count_not_toggled: Appliances that are turned off. count_closed: Containers/doors that are closed. count_not_sliced: Food items that require slicing.
Spatial	count_not_onfloor: Objects not placed on the floor layer. count_isolated: Objects with no neighbors of the same type. count_not_inside_target: Objects not contained within a specific furniture. count_not_ontop: Objects not placed on top of a specific surface. count_not_near_target: Objects not within 1 step of a target type.
Task-Specific	count_incomplete_salad: Plates lacking necessary ingredients. count_not_complete_tables: Tables lacking specific setups (e.g., candles).

to the nearest object of a specific type.

– *Conditions*: $=0$ (adjacent), >0 , increase, decrease, change.

- **Count(type, condition, function)**: Counts the number of objects of a specific type that satisfy a filter function.

– *Conditions*: $=0$, >0 , increase, decrease.

E.2 Count Functions

A diverse set of counting functions is implemented to support complex logical conditions (e.g., “all shoes are clean” is equivalent to *count_not_cleaned* == 0).

E.3 Reference Plan Generation and Task Complexity Analysis

While the original MINI-BEHAVIOR framework provides the high-level skeletons for household activities, it does not provide reference plans by default. Thus, we utilize the BFWS (Best-First Width Search) classical planner to generate reference plans for all 20 tasks in MINI-BEHAVIOR-GRAN. These reference plans serve as the ground truth for our agents and allow us to quantitatively categorize the complexity of each task.

Detailed statistics regarding the specific rule sets used for instruction generation and the resulting natural language statistics are discussed in the next section.

F Rule Set and Natural Language Prompt Examples For Different Tasks

We present the width-0 rule set for 10 over 20 household tasks in the Mini-BEHAVIOR-Gran environment. For $w > 0$, please refer to the codebase at [mini-behavior-gran/mini_behavior_utils/policy_sketch/](https://github.com/mini-behavior-gran/mini_behavior_utils/policy_sketch/).

F.1 Task 1: Laying Wood Floors

Features:

- H : holding an isolated wood (plywood) 1123
 - p : distance to the nearest isolated wood 1124
 - t : distance to an arbitrary (non-held) wood we’ll place next to 1125
 - n : number of isolated woods 1127
- $\{\neg H, p > 0, n > 0\} \mapsto \{p \downarrow, t?\}$; move close to isolated wood
 $\{\neg H, p = 0, n > 0\} \mapsto \{H\}$; pick up isolated wood when reachable 1128
 $\{H, t > 0, n > 0\} \mapsto \{t \downarrow\}$; move to some wood location
 $\{H, n > 0, t = 0\} \mapsto \{H?, n \downarrow, p?\}$; drop the isolated wood next to other wood

F.2 Task 2: Preparing Salad

Features:

- H_n : holding a not-sliceable food 1131
- H_s : holding a sliceable food 1132
- H_k : holding a slicer 1133
- U : number of sliceable food that is not sliced yet 1134
- d_p : distance to the selected incomplete plate p 1136
- d_n, d_s : distances to nearest not-sliceable / sliceable 1137
- k : distance to a slicer (knife) 1138
- B : selected plate p complete bottom salad 1140
- n : number of incomplete plates 1141
- nn : number of plates that do not have bottom salad yet 1142

- $\{U > 0, \neg H_k, k > 0\} \mapsto \{k \downarrow\}$; move to slicer
 $\{U > 0, \neg H_k, k = 0\} \mapsto \{H_k\}$; hold a slicer
Acquire slicer: $\{U > 0, H_k, d_s > 0\} \mapsto \{d_s \downarrow\}$; move to sliceable food 1144
 $\{U > 0, H_k, d_s = 0\} \mapsto \{U \downarrow, \neg H_k\}$; cut sliceable food
 $\{U = 0, H_k\} \mapsto \{\neg H_k\}$; put down slicer
 $\{n > 0, U = 0, nn > 0, \neg B, \neg H_n, d_n > 0\} \mapsto \{d_n \downarrow\}$
Place bottom salad: $\{n > 0, U = 0, nn > 0, \neg B, \neg H_n, d_n = 0\} \mapsto \{H_n\}$ 1145
 $\{n > 0, U = 0, nn > 0, \neg B, H_n, d_p > 0\} \mapsto \{d_p \downarrow\}$
 $\{n > 0, U = 0, nn > 0, \neg B, H_n, d_p = 0\} \mapsto \{B, nn \downarrow, \neg H_n\}$
 $\{nn = 0, n > 0, U = 0, \neg H_s, d_s > 0\} \mapsto \{d_s \downarrow\}$; move to sliceable food
Handle sliceable salad: $\{nn = 0, n > 0, U = 0, \neg H_s, d_s = 0\} \mapsto \{H_s\}$; pick up sliceable food 1146
 $\{nn = 0, n > 0, U = 0, H_s, d_p > 0\} \mapsto \{d_p \downarrow\}$; move to the plate
 $\{nn = 0, n > 0, U = 0, H_s, d_p = 0\} \mapsto \{n \downarrow, \neg H_s\}$; put down sliced food

F.3 Task 3: Cleaning Up the Kitchen Only

Features:

- *Holding flags*: H_b (blender), H_f (food), H_s (soap), H_r (rag), H_p (plate), H_o (vegetable oil) 1149
- *Distances*: d_{ct} (countertop), d_{fr} (fridge), d_{si} (sink), d_{c1} (cabinet for plate), d_{c2} (cabinet for oil), d_x (to object x) 1150
- *State booleans*: $\text{Open}(c)$, $\text{Tog}(s)$, Soaked , $\text{DustFree}(c)$, $\text{Clean}(p)$, $\text{OilIn}(c_2)$, Diff 1151
- *Counters*: N_b (blenders not on countertop), N_a (apples not in fridge), N_{cas} (casseroles not in fridge), N_s (soaps not next to sink), N_{cab} (cabinets not dusted), N_r (rags not next to sink), N_{p_clean} (plates not clean), N_{oil} (oil not in cabinet), N_{p_store} (plates not stored), N_{close} (closed furniture) 1152

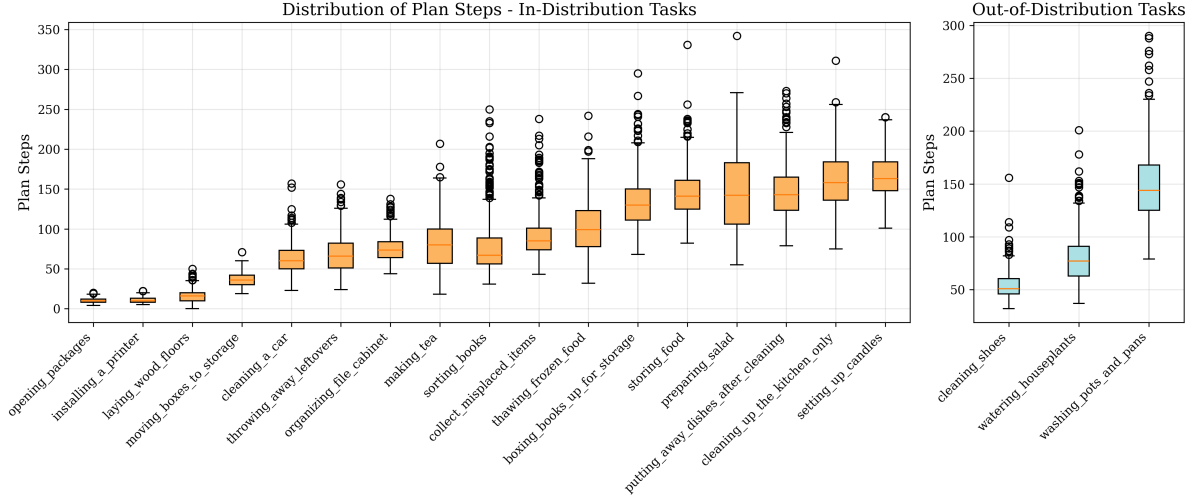


Figure 10: Distribution of reference plan steps required for tasks in MINI-BEHAVIOR-GRAN obtained by BWFS classical planner.

1164 *Open Cabinet and Fridge:* $\{N_{close} > 0, d_c > 0\} \mapsto \{d_c \downarrow\}$; move toward closed furniture
1165 $\{N_{close} > 0, d_c = 0\} \mapsto \{N_{close} \downarrow\}$
 $\{N_{close} = 0, \neg H_o, d_{oil} > 0\} \mapsto \{d_{oil} \downarrow\}$; approach oil
 $\{N_{close} = 0, \neg H_o, d_{oil} = 0\} \mapsto \{H_o\}$; pick up oil
1166 *Oil placement:* $\{N_{close} = 0, H_o, N_{oil} > 0, d_{c2} > 0\} \mapsto \{d_{c2} \downarrow\}$; go to cabinet c_2
 $\{N_{close} = 0, H_o, N_{oil} > 0, d_{c2} = 0\} \mapsto \{N_{oil} \downarrow, \neg H_o\}$; place oil in c_2
 $\{\neg \text{Tog}(s), d_{si} > 0\} \mapsto \{d_{si} \downarrow\}$; toggle sink on
 $\{\neg \text{Tog}(s), d_{si} = 0\} \mapsto \{\text{Tog}(s)\}$
 $\{\neg H_r, d_{rag} > 0\} \mapsto \{d_{rag} \downarrow\}$; get rag
 $\{\neg H_r, d_{rag} = 0\} \mapsto \{H_r\}$
1167 *Plate cleaning:* $\{H_r, \neg \text{Soaked}, d_{si} > 0\} \mapsto \{d_{si} \downarrow\}$; soak rag
 $\{H_r, \neg \text{Soaked}, d_{si} = 0\} \mapsto \{\text{Soaked}\}$
 $\{H_r, \text{Soaked}, N_{p_clean} > 0, d_{plate} > 0\} \mapsto \{d_{plate} \downarrow\}$
 $\{H_r, \text{Soaked}, N_{p_clean} > 0, d_{plate} = 0\} \mapsto \{N_{p_clean} \downarrow, d_{plate}?\}$
 $\{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, \neg H_p, N_{p_store} > 0, d_{plate} > 0\} \mapsto \{d_{plate} \downarrow\}$
Plate placement distinct from oil: $\{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, H_p, N_{p_store} > 0, d_{c1} > 0\} \mapsto \{d_{c1} \downarrow\}$
 $\{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, H_p, N_{p_store} > 0, d_{c1} = 0\} \mapsto \{N_{p_store} \downarrow, \neg H_p\}$
 $\{N_b > 0, \neg H_b, d_{blender} > 0\} \mapsto \{d_{blender} \downarrow\}$
1168 *Blender to countertop:* $\{N_b > 0, \neg H_b, d_{blender} = 0\} \mapsto \{H_b\}$
 $\{H_b, N_b > 0, d_{ct} > 0\} \mapsto \{d_{ct} \downarrow\}$
 $\{H_b, N_b > 0, d_{ct} = 0\} \mapsto \{N_b \downarrow, \neg H_b\}$
 $\{N_{close} = 0, \neg H_f, d_{apple} > 0\} \mapsto \{d_{apple} \downarrow\}$
 $\{N_{close} = 0, \neg H_f, d_{apple} = 0\} \mapsto \{H_f\}$
1169 *Apple to fridge:* $\{N_{close} = 0, H_f, N_a > 0, d_{fr} > 0\} \mapsto \{d_{fr} \downarrow\}$
 $\{N_{close} = 0, H_f, N_a > 0, d_{fr} = 0\} \mapsto \{N_a \downarrow, \neg H_f\}$
 $\{N_{close} = 0, \neg H_f, d_{casserole} > 0\} \mapsto \{d_{casserole} \downarrow\}$
 $\{N_{close} = 0, \neg H_f, d_{casserole} = 0\} \mapsto \{H_f\}$
1170 *Casserole to fridge:* $\{N_{close} = 0, H_f, N_{cas} > 0, d_{fr} > 0\} \mapsto \{d_{fr} \downarrow\}$
 $\{N_{close} = 0, H_f, N_{cas} > 0, d_{fr} = 0\} \mapsto \{N_{cas} \downarrow, \neg H_f\}$
 $\{\neg H_r, d_{rag} > 0\} \mapsto \{d_{rag} \downarrow\}$; get rag
 $\{\neg H_r, d_{rag} = 0\} \mapsto \{H_r\}$
1171 *Cabinet dusting:* $\{H_r, N_{cab} > 0, d_c > 0\} \mapsto \{d_c \downarrow\}$; clean cabinet
 $\{H_r, N_{cab} > 0, d_c = 0\} \mapsto \{N_{cab} \downarrow, \neg H_r\}$
 $\{N_{cab} = 0, N_{p_clean} = 0, \neg H_r, d_{rag} > 0\} \mapsto \{d_{rag} \downarrow\}$
 $\{N_{cab} = 0, N_{p_clean} = 0, \neg H_r, d_{rag} = 0\} \mapsto \{H_r\}$
1172 *Rag near/inside sink:* $\{N_{cab} = 0, N_{p_clean} = 0, H_r, N_r > 0, d_{si} > 0\} \mapsto \{d_{si} \downarrow\}$
 $\{N_{cab} = 0, N_{p_clean} = 0, H_r, N_r > 0, d_{si} = 0\} \mapsto \{N_r \downarrow, \neg H_r\}$
 $\{N_{cab} = 0, N_{p_clean} = 0, \neg H_s, d_{soap} > 0\} \mapsto \{d_{soap} \downarrow\}$
 $\{N_{cab} = 0, N_{p_clean} = 0, \neg H_s, d_{soap} = 0\} \mapsto \{H_s\}$
1173 *Soap next to sink:* $\{N_{cab} = 0, N_{p_clean} = 0, H_s, N_s > 0, d_{si} > 0\} \mapsto \{d_{si} \downarrow\}$
 $\{N_{cab} = 0, N_{p_clean} = 0, H_s, N_s > 0, d_{si} = 0\} \mapsto \{N_s \downarrow, \neg H_s\}$

1174 F.4 Task 4: Organizing File Cabinet

1175 Features:

- H_m : holding a marker
- H_f : holding a folder
- H_d : holding a document
- p_m : distance to the nearest marker
- p_f : distance to the nearest folder
- p_d : distance to the nearest document
- p_t : distance to a table
- p_c : distance to a valid cabinet target
- N_m : number of markers not on a table
- N_f : number of folders not in a cabinet
- N_d : number of documents not in a cabinet
- N_o : number of closed cabinets

1188 *Opening Cabinet:* $\{N_o > 0, p_c > 0\} \mapsto \{p_c \downarrow\}$; move toward a closed cabinet
 $\{N_o > 0, p_c = 0\} \mapsto \{N_o \downarrow\}$; open the cabinet when reachable
 $\{N_m > 0, \neg H_m, p_m > 0\} \mapsto \{p_m \downarrow, p_t?\}$; move toward the nearest marker
 $\{N_m > 0, \neg H_m, p_m = 0\} \mapsto \{H_m, p_t?\}$; pick up the marker when reachable
1189 *Marker task:* $\{N_m > 0, H_m, p_t > 0\} \mapsto \{p_t \downarrow\}$; move toward a table
 $\{N_m > 0, H_m, p_t = 0\} \mapsto \{N_m \downarrow, \neg H_m, p_m?\}$; put marker on table
 $\{N_f > 0, N_o = 0, \neg H_f, p_f > 0\} \mapsto \{p_f \downarrow, p_c?\}$
 $\{N_f > 0, N_o = 0, \neg H_f, p_f = 0\} \mapsto \{H_f, p_c?\}$
1190 *Folder task:* $\{N_f > 0, N_o = 0, H_f, p_c > 0\} \mapsto \{p_c \downarrow\}$
 $\{N_f > 0, N_o = 0, H_f, p_c = 0\} \mapsto \{N_f \downarrow, \neg H_f, p_f?\}$
 $\{N_d > 0, N_o = 0, \neg H_d, p_d > 0\} \mapsto \{p_d \downarrow, p_c?\}$
 $\{N_d > 0, N_o = 0, \neg H_d, p_d = 0\} \mapsto \{H_d, p_c?\}$
1191 *Document task:* $\{N_d > 0, N_o = 0, H_d, p_c > 0\} \mapsto \{p_c \downarrow\}$
 $\{N_d > 0, N_o = 0, H_d, p_c = 0\} \mapsto \{N_d \downarrow, \neg H_d, p_d?\}$

1192 F.5 Task 5: Thawing Frozen Food

1193 Features:

- H_{fish} : holding a fish
- H_{olive} : holding an olive
- H_{date} : holding a date
- p_{fish} : distance to the nearest fish
- p_{olive} : distance to the nearest olive
- p_{date} : distance to the nearest date
- p_{sink} : distance to a sink

Task	Split	Class	Goal Description	Mean Steps
opening_packages	ID	short	Open every package.	10.2
installing_a_printer	ID	short	Place a printer on a table and turn it on.	10.6
laying_wood_floors	ID	short	Lay every plywood sheet on the floor, each touching at least one other sheet.	15.9
moving_boxes_to_storage	ID	short	Place every carton inside a shelf.	36.3
cleaning_a_car	ID	short	Make at least one car dust-free. Place a rag and soap together inside a bucket.	62.6
throwing_away_leftovers	ID	middle	Throw away every hamburger by placing it in a trash can.	68.3
organizing_file_cabinet	ID	middle	Put a marker on a table and file every folder and document in a cabinet.	75.5
making_tea	ID	middle	Slice a lemon. Put a teapot on an activated stove. Keep a soaked teabag with the teapot.	78.7
sorting_books	ID	middle	Place every book and every hardback on a shelf.	78.8
collect_misplaced_items	ID	middle	Place all gym shoes, necklaces, notebooks, and socks on a table.	90.7
thawing_frozen_food	ID	middle	Place a date label next to a fish. Put every fish next to a sink. Set an olive next to a sink.	101.7
boxing_books_up_for_storage	ID	long	Place every book inside a box.	134.4
storing_food	ID	long	Store every cookable item inside a cabinet.	144.6
preparing_salad	ID	long	Slice all apples and tomatoes; place them on plates. Put every radish and lettuce on plates. Ensure each plate holds at least one cookable item.	146.7
putting_away_dishes_after_cleaning	ID	long	Put every plate inside a cabinet.	147.7
cleaning_up_the_kitchen_only	ID	long	Put every blender on a countertop. Refrigerate all apples and casseroles. Keep soap next to a sink. Make plates unstained and cabinets dust-free. Place each rag either beside or inside the sink. Store plates and vegetable oil in different cabinets.	161.0
setting_up_candles	ID	long	On every table, place three distinct candles on top.	165.2
cleaning_shoes	OOD	short	Leave a towel on the floor and make sure every shoe is unstained and dust-free.	54.2
watering_houseplants	OOD	middle	Water every potted plant until it is soaked.	79.4
washing_pots_and_pans	OOD	long	Clean every teapot, kettle, and pan so they're unstained, then store each inside a cabinet.	149.5

Table 6: Classification of tasks in MINI-BEHAVIOR-GRAN based on the median number of plan steps, as determined by the BWFS planner. Tasks are categorized into short, middle, and long classes to reflect varying planning complexity based on plan length.

1201	• N_{fish} : number of fish not next to a sink	• p_t : distance to the nearest teapot	1215
1202	• N_{olive} : number of olives not next to a sink	• p_{tb} : distance to the nearest tea bag	1216
1203	• N_{date} : number of dates not next to a fish	• p_s : distance to the stove	1217
		• p_{sink} : distance to the sink	1218
		• is_sliced(lemon): whether the lemon is sliced	1219
1204	<i>Fish Task:</i> $\{\neg H_{fish}, p_{fish} > 0\} \mapsto \{p_{fish} \downarrow\}$	• is_toggled(stove): whether the stove is toggled on	1220
	$\{\neg H_{fish}, p_{fish} = 0\} \mapsto \{H_{fish}\}$	• is_toggled(sink): whether the sink is toggled on	1221
	$\{H_{fish}, p_{sink} > 0\} \mapsto \{p_{sink} \downarrow\}$	• is_soaked(tea_bag): whether the tea bag is soaked	1222
	$\{H_{fish}, p_{sink} = 0\} \mapsto \{N_{fish} \downarrow, \neg H_{fish}, p_{fish}?\}$	• atsamelocation(tea_bag, teapot): tea bag at same location as teapot	1223
	$\{\neg H_{olive}, p_{olive} > 0\} \mapsto \{p_{olive} \downarrow\}$	• onTop(teapot, stove): teapot on stove	1224
1205	<i>Olive Task:</i> $\{\neg H_{olive}, p_{olive} = 0\} \mapsto \{H_{olive}\}$		1225
	$\{H_{olive}, p_{sink} > 0\} \mapsto \{p_{sink} \downarrow\}$		1226
	$\{H_{olive}, p_{sink} = 0\} \mapsto \{N_{olive} \downarrow, \neg H_{olive}, p_{olive}?\}$		1227
	$\{\neg H_{date}, N_{fish\ olive} = 0, p_{date} > 0\} \mapsto \{p_{date} \downarrow\}$		1228
1206	<i>Date Task:</i> $\{\neg H_{date}, N_{fish\ olive} = 0, p_{date} = 0\} \mapsto \{H_{date}\}$		
	$\{H_{date}, N_{fish\ olive} = 0, p_{fish} > 0\} \mapsto \{p_{fish} \downarrow\}$		
	$\{H_{date}, N_{fish\ olive} = 0, p_{fish} = 0\} \mapsto \{N_{date} \downarrow, \neg H_{date}, p_{date}?\}$		

F.6 Task 6: Making Tea

Features:

1207	• H_l : holding a lemon		
1208	• H_{knife} : holding a knife		
1209	• H_t : holding a teapot		
1210	• H_{tb} : holding a tea bag		
1211	• p_l : distance to the nearest lemon		
1212	• p_{knife} : distance to the knife		
1213		<i>Toggle stove:</i> $\{\neg \text{is_toggled}(\text{stove}), p_s > 0\} \mapsto \{p_s \downarrow\}$	1229
1214		$\{\neg \text{is_toggled}(\text{stove}), p_s = 0\} \mapsto \{\text{is_toggled}(\text{stove})\}$	
		$\{\neg H_{knife}, p_{knife} > 0, \neg \text{is_sliced}(\text{lemon})\} \mapsto \{p_{knife} \downarrow\}$	
		$\{\neg H_{knife}, p_{knife} = 0, \neg \text{is_sliced}(\text{lemon})\} \mapsto \{H_{knife}\}$	
		<i>Slice lemon:</i> $\{H_{knife}, \neg \text{is_sliced}(\text{lemon}), p_l > 0\} \mapsto \{p_l \downarrow\}$	1230
		$\{H_{knife}, \neg \text{is_sliced}(\text{lemon}), p_l = 0\} \mapsto \{\text{is_sliced}(\text{lemon})\}$	
		$\{H_{knife}, \text{is_sliced}(\text{lemon})\} \mapsto \{\neg H_{knife}, p_{knife}?\}$	

$\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \neg \text{onTop}(\text{tp}, \text{st}), \neg H_t, p_t > 0\} \mapsto \{p_t \downarrow\}$
 $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \neg \text{onTop}(\text{tp}, \text{st}), \neg H_t, p_t = 0\} \mapsto \{H_t\}$
 $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \neg \text{onTop}(\text{tp}, \text{st}), H_t, p_s > 0\} \mapsto \{p_s \downarrow\}$
 $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \neg \text{onTop}(\text{tp}, \text{st}), H_t, p_s = 0\} \mapsto \{\text{onTop}(\text{tp}, \text{st}), \neg H_t, p_t?\}$
Handle teapot: $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \text{onTop}(\text{tp}, \text{st}), \neg H_{tb}, p_{tb} > 0\} \mapsto \{p_{tb} \downarrow\}$
 $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \text{onTop}(\text{tp}, \text{st}), \neg H_{tb}, p_{tb} = 0\} \mapsto \{H_{tb}\}$
 $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \text{onTop}(\text{tp}, \text{st}), H_{tb}, p_t > 0\} \mapsto \{p_t \downarrow\}$
 $\{H_{tb}, \neg \text{atsameloc}(\text{tb}, \text{tp}), \text{onTop}(\text{tp}, \text{st}), p_t = 0\} \mapsto \{\text{atsameloc}(\text{tb}, \text{tp}), \neg H_{tb}, p_{tb}?\}$

F.7 Task 7: Opening Packages

Features:

- p : distance to the nearest package
- N : number of unopened packages

$\{N > 0, p > 0\} \mapsto \{p \downarrow\}$; move toward the nearest unopened package
 $\{N > 0, p = 0\} \mapsto \{N \downarrow\}$; open the package when reachable

F.8 Task 8: Boxing Books Up for Storage

Features:

- H : holding an unboxed book
- p : distance to the nearest unboxed book
- b : distance to a valid box target
- N : number of unboxed books

$\{\neg H, p > 0\} \mapsto \{p \downarrow, b?\}$; move toward the nearest unboxed book
 $\{\neg H, p = 0\} \mapsto \{H\}$; pick up the book when reachable
 $\{H, N > 0, b > 0\} \mapsto \{b \downarrow\}$; move toward a chosen box spot
 $\{H, N > 0, b = 0\} \mapsto \{\neg H, N \downarrow, p?\}$; place the book into the box

F.9 Task 9: Collect Misplaced Items

Features:

- H_s : holding a gym shoe
- H_n : holding a necklace
- H_b : holding a notebook
- H_k : holding a sock
- p_s : distance to the nearest gym shoe
- p_n : distance to the nearest necklace
- p_b : distance to the nearest notebook
- p_k : distance to the nearest sock
- p_t : distance to a table
- N_s : number of gym shoes not on a table
- N_n : number of necklaces not on a table
- N_b : number of notebooks not on a table
- N_k : number of socks not on a table

$\{\neg H_s, N_s > 0, p_s > 0\} \mapsto \{p_s \downarrow, p_t?\}$
 $\{\neg H_s, N_s > 0, p_s = 0\} \mapsto \{H_s, p_t?\}$
Gym shoe task: $\{H_s, N_s > 0, p_t > 0\} \mapsto \{p_t \downarrow\}$
 $\{H_s, N_s > 0, p_t = 0\} \mapsto \{N_s \downarrow, \neg H_s, p_s?\}$
 $\{\neg H_n, N_n > 0, p_n > 0\} \mapsto \{p_n \downarrow, p_t?\}$
Necklace task: $\{\neg H_n, N_n > 0, p_n = 0\} \mapsto \{H_n, p_t?\}$
 $\{H_n, N_n > 0, p_t > 0\} \mapsto \{p_t \downarrow\}$
 $\{H_n, N_n > 0, p_t = 0\} \mapsto \{N_n \downarrow, \neg H_n, p_n?\}$
 $\{\neg H_b, N_b > 0, p_b > 0\} \mapsto \{p_b \downarrow, p_t?\}$
Notebook task: $\{\neg H_b, N_b > 0, p_b = 0\} \mapsto \{H_b, p_t?\}$
 $\{H_b, N_b > 0, p_t > 0\} \mapsto \{p_t \downarrow\}$
 $\{H_b, N_b > 0, p_t = 0\} \mapsto \{N_b \downarrow, \neg H_b, p_b?\}$

$\{\neg H_k, N_k > 0, p_k > 0\} \mapsto \{p_k \downarrow, p_t?\}$

Sock task: $\{\neg H_k, N_k > 0, p_k = 0\} \mapsto \{H_k, p_t?\}$

$\{H_k, N_k > 0, p_t > 0\} \mapsto \{p_t \downarrow\}$

$\{H_k, N_k > 0, p_t = 0\} \mapsto \{N_k \downarrow, \neg H_k, p_k?\}$

F.10 Task 10: Putting Away Dishes After Cleaning

Features:

- H : holding a plate
- p : distance to the nearest plate
- b : distance to a valid cabinet target
- N : number of unboxed plates
- is_opened : whether the cabinet is opened

$\{\neg \text{is_opened}, b > 0\} \mapsto \{b \downarrow\}$; move toward the cabinet
 $\{\neg \text{is_opened}, b = 0\} \mapsto \{\text{is_opened}\}$; open the cabinet when reachable
 $\{\neg H, N > 0, p > 0, \text{is_opened}\} \mapsto \{p \downarrow, b?\}$
 $\{\neg H, N > 0, p = 0, \text{is_opened}\} \mapsto \{H, b?\}$
 $\{H, N > 0, b > 0, \text{is_opened}\} \mapsto \{b \downarrow\}$
 $\{H, N > 0, b = 0, \text{is_opened}\} \mapsto \{\neg H, N \downarrow, p?\}$

F.11 Instruction Generation Statistics

We used a template-based generator to translate the symbolic rule sets into natural language instructions, ensuring alignment with the VLA’s training data format. Table 7 presents the summary statistics for the generated instructions across the three granularity levels.

G Implementation Details

G.1 Model Architecture

Our policy is built upon the OpenVLA architecture (Kim et al., 2024). Specifically, we utilize the Prismatic-7B backbone (Karamcheti et al., 2024), which integrates a frozen Llama-2-7B language model with fused visual features. The visual encoder consists of a dual-stream setup combining SigLIP (for semantic understanding) and DINOv2 (for fine-grained spatial geometry) (Zhai et al., 2023; Oquab et al., 2024).

G.2 Training Configuration

We fine-tune the model using Low-Rank Adaptation (LoRA) on the language model backbone, keeping the visual encoders frozen. All training setting variants (Pure, Centroid, Axial) share the same underlying training hyperparameters to ensure fair comparison.

Hardware and Duration: Training was conducted on a single node equipped with $8 \times$ NVIDIA A100 GPUs. A standard training run for 100,000 steps took approximately 23 hours to complete.

Table 7: Statistics of generated natural language instructions across granularity levels.

Granularity	Count	Avg Length (words)	Std Length	Min	Max	Unique Tokens	TTR (Type-Token Ratio)	Avg Sent. Length
Fine (F)	1636	41.12	11.36	19	87	140	0.0021	41.12
Medium (M)	859	38.62	12.76	22	81	108	0.0033	38.62
Coarse (C)	587	33.60	13.39	22	95	94	0.0048	33.60

Table 8 summarizes the core hyperparameters used across our experiments.

Table 8: Hyperparameters for OpenVLA fine-tuning on MINI-BEHAVIOR-GRAN.

Hyperparameter	Value
Base Model	Prismatic-7B (Llama-2)
Visual Encoders	SigLIP + DINOv2
Parameter Efficient Tuning	LoRA (Rank $r = 32$)
Compute Hardware	8× NVIDIA A100
Training Duration	~23 hours
Optimization Steps	100,000
Steps Before Decay	80,000
Learning Rate	2.5×10^{-4}
Batch Size (per GPU)	7 – 8

We also train a lightweight VLM predictor to quantify instruction predictability via perplexity. The model is fine-tuned using Low-Rank Adaptation (LoRA) on the Qwen3-VL-4B-Instruct backbone, with vision encoders frozen to preserve visual grounding capabilities established during pretraining. All models are trained on MINI-BEHAVIOR-GRAN instruction and image training data pairs, with training and evaluation splits maintained consistently across experiments. Hardware and Duration: Training was conducted on a single node equipped with 4× NVIDIA RTX 5090 GPUs. Each training run for 2 epochs. Table 9 summarizes the core hyperparameters used for VLM predictor training.

Table 9: Hyperparameters for Qwen3-VL-4B-Instruct fine-tuning on MINI-BEHAVIOR-GRAN (High-domain).

Hyperparameter	Value
Base Model	Qwen3-VL-4B-Instruct
Visual Encoders	Frozen (Qwen3-VL native)
Parameter Efficient Tuning	LoRA (Rank $r = 64$, $\alpha = 128$)
LoRA Target Modules	All linear layers
Compute Hardware	4× NVIDIA RTX 5090
Training Epochs	2
Learning Rate	1.0×10^{-5}
Learning Rate Scheduler	Cosine with 10% warmup
Batch Size (per GPU)	32
Gradient Accumulation Steps	1
DeepSpeed Strategy	Zero-2

H Additional Experimental Results

This section provides additional detailed per-task performance breakdowns, and failure mode distributions.

H.1 Detailed Task Performance

Figure 11 details the success rates across all 20 tasks in the benchmark. We see some tasks are too difficult to solve and thus low across all instruction granularity. And then we also see the U shape trends in most tasks, showing the same phenomenon we discussed in (Q1) that both very fine and very coarse instructions are easier for VLA to follow.

H.2 Failure Analysis

Figure 12 presents the distribution of failure types across varying instruction widths. It confirms that low width encounters regression failures more frequently, while high width predominantly leads to stagnation failures. This aligns with our earlier observations in (Q3) regarding the distinct failure modes associated with different instruction granularities.

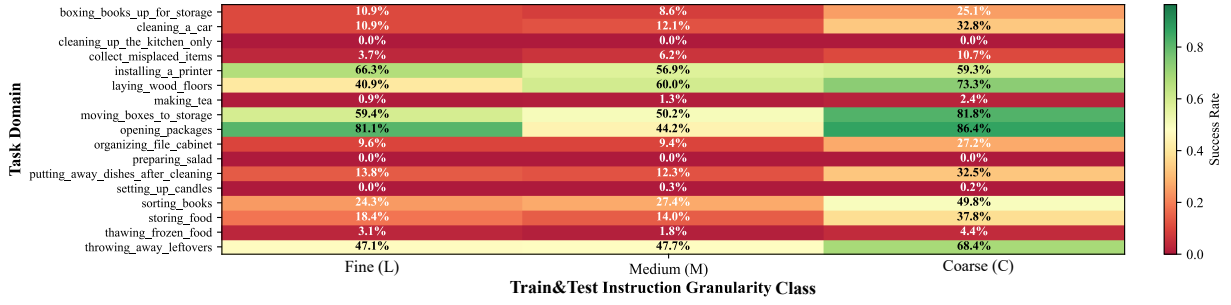


Figure 11: Detailed per-task success rates.

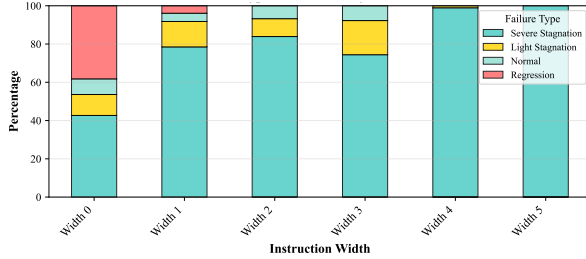


Figure 12: Failure type distribution across instruction widths.

H.3 VLA Transformer Attention Weight Visualization

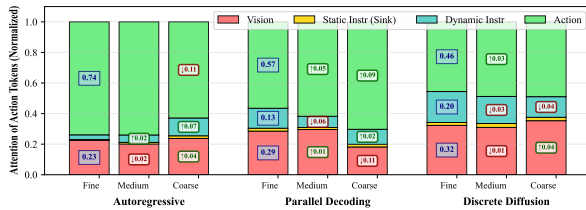


Figure 13: Attention from action tokens. Self-attention dominates, followed by vision; No clear trend across granularity levels.

Attention analysis (Figure 13) reveals there is no obvious reduction in language-token attention. This suggests that in VLA architectures, maintained attention weights do not necessarily equate to language grounding, highlighting a complex dynamic that requires future investigation.

Nevertheless, a closer inspection of the attention distributions reveals distinct architectural behaviors and compensatory mechanisms.

Autoregressive Models (Compensatory Reading). Autoregressive generation exhibits a heavy baseline reliance on historical action tokens (accounting for 0.74 of normalized attention in the Fine setting). Interestingly, when presented with Coarse instructions, attention to the dynamic instruction increases ($\uparrow 0.07$) while action attention

decreases ($\downarrow 0.11$). This suggests a compensatory mechanism: as the rule width w increases and immediate subgoals disappear, the model attempts to ground more heavily on the language prompt to search for clues.

Parallel Decoding (Attending more on action modality). This architecture demonstrates a stark multimodal decoupling under coarse instructions. The attention paid to visual tokens drops significantly ($\downarrow 0.11$) in the Coarse setting, accompanied by an increased reliance on generated action tokens ($\uparrow 0.09$). This could imply that without detailed instruction guidance at each step, parallel decoding intend to trust more on action history to decide the next actions, showing a different strategy from autoregressive models.

Discrete Diffusion (Structural Stability). Diffusion models exhibit the most balanced attention distribution, maintaining the highest baseline attention to both dynamic instructions (0.20) and visual features (0.32). Notably, this distribution remains remarkably rigid regardless of instruction granularity (with fluctuations bounded within ± 0.04).

H.4 Sample Generated Instructions by Granularity

To illustrate the qualitative differences between granularity levels, we provide five random samples for each category below. Note how Low (L) granularity explicitly specifies navigation and preconditions (e.g., “move toward,” “not yet at”), while High (H) granularity focuses almost exclusively on the final state change (e.g., “reduce the count of...”), shown in Table 10.

Fine Granularity (F) Samples

1. At step 11: When the count of uncleaned plate is above 0, not yet at sink 0 and sink is off, please try to move toward sink 0.
2. At step 8: When not yet at soap 0, not holding soap 0, the count of unwiped car is 0 and the count of rag not inside bucket is 0, please try to move toward soap 0.
3. At step 27: When the count of chip and oatmeal sugar and vegetable oil not inside cabinet is above 0, you are at oatmeal 1 and not holding oatmeal 1, please pick up oatmeal 1.
4. At step 15: When the count of plate not inside cabinet is above 0, you are at plate 3, the count of closed cabinet is 0 and not holding plate 3, please pick up plate 3.
5. At step 13: When the count of fish and olive not near sink is above 0, not yet at olive 0 and not holding olive 0, please try to move toward olive 0.

Medium Granularity (M) Samples

1. At step 2: When the count of plate not inside cabinet is above 0, the count of closed cabinet is 0 and not holding plate 2, please pick up plate 2.
2. At step 6: When the count of uncleaned plate is above 0, the count of dry rag is above 0 and holding rag 0, please reduce the count of dry rag.
3. At step 9: When the count of plate not inside cabinet 0 is above 0, the count of closed cabinet is 0, the count of vegetable oil not inside cabinet 1 is 0, the count of uncleaned plate is 0 and not holding plate 0, please pick up plate 0.
4. At step 13: When the count of book not inside box is above 0 and not holding book 2, please pick up book 2.
5. At step 9: When the count of chip and oatmeal sugar and vegetable oil not inside cabinet is above 0, holding chip 1 and the count of closed cabinet is 0, please not holding chip 1, then reduce the count of chip and oatmeal sugar and vegetable oil not inside cabinet.

Coarse Granularity (C) Samples

1. At step 2: When the count of hamburger not inside ashcan is above 0, please reduce the count of hamburger not inside ashcan.
2. At step 5: When the count of plate not inside cabinet is above 0 and the count of closed cabinet is 0, please reduce the count of plate not inside cabinet.
3. At step 3: When the count of teapot not on stove is above 0, please reduce the count of teapot not on stove.
4. At step 9: When the count of plate not inside cabinet is above 0 and the count of closed cabinet is 0, please reduce the count of plate not inside cabinet.
5. At step 8: When the count of chip and oatmeal sugar and vegetable oil not inside cabinet is above 0 and the count of closed cabinet is 0, please reduce the count of chip and oatmeal sugar and vegetable oil not inside cabinet.

Table 10: Sample generated instructions across granularity levels.